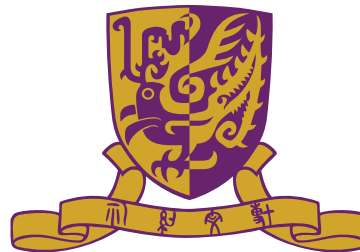


DDA5001 Machine Learning

Unsupervised Learning:
Dimensionality Reduction (PCA) &
Clustering (k-means)

Xiao Li

School of Data Science
The Chinese University of Hong Kong, Shenzhen



Recap: PCA

PCA modeling:

$$\mathbf{x}_i = \mathbf{A}\boldsymbol{\theta}_i + \boldsymbol{\mu}$$

Interpretation: Find the **closest** k -dimensional data point $\tilde{\mathbf{x}}_i = \mathbf{A}\boldsymbol{\theta}_i = \mathbf{A}\mathbf{A}^\top \mathbf{x}_i$ to $\mathbf{x}_i$ (projecting $\mathbf{x}_i$ onto the k -dimensional subspace spanned by the columns of $\mathbf{A}$).

Process of applying PCA:

- ▶ Remove the mean: $\mathbf{x}_i = \mathbf{x}_i - \boldsymbol{\mu}$, where $\boldsymbol{\mu} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i$.
- ▶ Compute SVD of the data matrix $\mathbf{X} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^\top$ (recommended) or eigen decomposition of the covariance matrix $\mathbf{S} = \mathbf{U}\boldsymbol{\Lambda}\mathbf{U}^\top$. ($\mathbf{S} = \mathbf{X}\mathbf{X}^\top$)
- ▶ Basis $\mathbf{A} = [\mathbf{u}_1, \dots, \mathbf{u}_k]$, principle component $\boldsymbol{\theta}_i = \mathbf{A}^\top \mathbf{x}_i$, and k -dimensional sample is $\tilde{\mathbf{x}}_i = \mathbf{A}\boldsymbol{\theta}_i = \mathbf{A}\mathbf{A}^\top \mathbf{x}_i$.

PCA — Continued

Clustering: k-means

PCA From the Matrix Factorization Perspective

Recall the (first form) PCA (we assume without loss of generality that $\mu = 0$)

$$\underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}, \{\boldsymbol{\theta}_i\}}{\text{minimize}} \quad \sum_{i=1}^n \|\mathbf{x}_i - \mathbf{A}\boldsymbol{\theta}_i\|_2^2.$$

$$\Theta = [\boldsymbol{\theta}_1 \cdots \boldsymbol{\theta}_n] \in \mathbb{R}^{k \times n}.$$

It can be written in a matrix form: $\mathbf{X} - \mathbf{A}\Theta = [\mathbf{x}_1 - \mathbf{A}\boldsymbol{\theta}_1 \cdots \mathbf{x}_n - \mathbf{A}\boldsymbol{\theta}_n]$

$$\underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}, \Theta}{\text{minimize}} \quad \underbrace{\|\mathbf{X} - \mathbf{A}\Theta\|_F^2}_{\in \mathbb{R}^{d \times n}} \Rightarrow \mathbf{X} \approx \mathbf{A}\Theta$$

→ Euclidean norm of matrix

where $\mathbf{A} \in \mathbb{R}^{d \times k}$ and $\Theta \in \mathbb{R}^{k \times n}$, and $\|\cdot\|_F$ is the **Frobenius norm**.

- ▶ The above problem is also called **low-rank matrix factorization**.
- ▶ **Interpretation:** Factorize $\mathbf{X}$ into two factors' multiplication, where the latter is a **low-rank matrix**.

PCA From the Matrix Factorization Perspective

Low-rank matrix factorization

$$\underset{A^T A = I, \Theta}{\text{minimize}} \quad \|X - A\Theta\|_F^2$$

- Calculate the SVD of $X = U\Sigma V^T$.

$$\Sigma = \begin{bmatrix} \sigma_1 & & & \\ & \ddots & & \\ & & \sigma_n & \\ & & & 0 \end{bmatrix}, \quad \sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_n \geq 0$$

Solution from PCA

- One optimal solution to the above (low-rank) matrix factorization problem is given by

$$A = [u_1, \dots, u_k], \quad \Theta = [\sigma_1 v_1, \dots, \sigma_k v_k]^T$$

- It has infinitely many equivalent optimal solutions.

$$\sim A\Phi\Phi^T\Theta, \text{ where } \Phi \text{ is orthogonal.}$$

It is a closed-form solution to a nonconvex optimization problem.

More General Matrix Factorization

We can also remove the orthogonal constraint on $\mathbf{A}$ to allow more flexibility

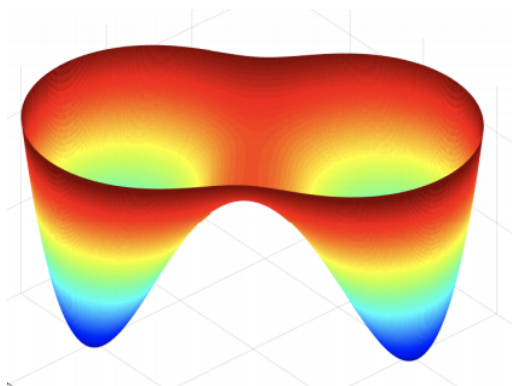
$$\underset{\mathbf{A} \in \mathbb{R}^{d \times k}, \mathbf{\Theta} \in \mathbb{R}^{k \times n}}{\text{minimize}} \quad \|\mathbf{X} - \mathbf{A}\mathbf{\Theta}\|_F^2.$$

One optimal solution to the above (low-rank) matrix factorization problem is given by

$$\mathbf{A} = [\sqrt{\sigma_1} \mathbf{u}_1, \dots, \sqrt{\sigma_k} \mathbf{u}_k], \quad \mathbf{\Theta} = [\sqrt{\sigma_1} \mathbf{v}_1, \dots, \sqrt{\sigma_k} \mathbf{v}_k]^\top,$$

and it has infinitely many equivalent optimal solution. $\underbrace{\mathbf{A} \mathbf{Q} \mathbf{Q}^\top}_{\tilde{\mathbf{A}}} \underbrace{\mathbf{\Theta}}_{\tilde{\mathbf{\Theta}}} = \mathbf{A} \mathbf{\Theta}$

- A nonconvex optimization problem



- Fortunately, closed-form solution exists.

LoRA: Low-rank Adaptation

In the **post-training** stage of large models (like large language models), we often need to learn an incremental to the learned model to incorporate new knowledge. That is

↓ pre-training

$$\underset{\Delta\Theta \in \mathbb{R}^{m \times n}}{\text{minimize}} \quad \mathcal{L}(\hat{\Theta} + \Delta\Theta)$$

Low-rank Adaptation (LoRA) uses the simple idea of PCA. It does not aim to learn the full $\Delta\Theta$, it instead learns a **PCA decomposition** of $\Delta\Theta$.

↓

$$\underset{A \in \mathbb{R}^{m \times r}, B \in \mathbb{R}^{r \times n}}{\text{minimize}} \quad \mathcal{L}(\hat{\Theta} + AB)$$

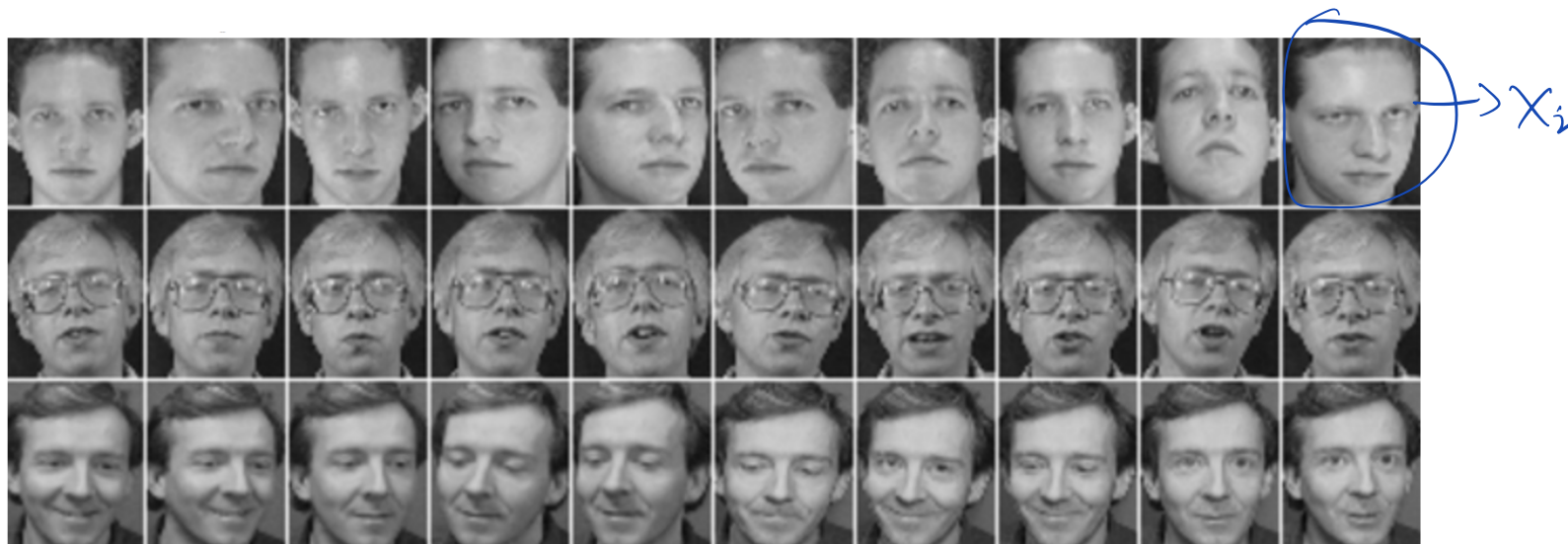
where $r \ll \min\{m, n\}$.

$$mn \rightarrow r(m+n)$$

- ▶ LoRA approach can **save computation memory**.
- ▶ It is now widely utilized in large language models.
- ▶ We will use LoRA in our final project.

Apply PCA to Real Image Dataset

The ORL Database of Faces



- ▶ 40 persons.
- ▶ Each has 10 distinct face images. $n = 40 \times 10 = 400$
- ▶ Each image is of size 92×112 . $= d$ of each x_i
- ▶ The images were taken at different times, varying the lighting, facial expressions (open / closed eyes, smiling / not smiling) and facial details (glasses / no glasses).

Form the Data Matrix

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \xrightarrow{\text{vec}} a = \begin{bmatrix} 1 \\ 3 \\ 2 \\ 4 \end{bmatrix}$$



92x112

$$\xrightarrow{\text{vectorize}} \mathbf{x}_i = \begin{bmatrix} \mathbf{x}_i[1] \\ \vdots \\ \mathbf{x}_i[92 \times 112] \end{bmatrix} \in \mathbb{R}^{10304}$$

- Do the same thing for all the face images, we get

$$\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_n] \in \mathbb{R}^{d \times n},$$

where $d = 10304$, $n = 10 \times 40 = 400$.

$d \gg n \xrightarrow{\text{PCA}} d \rightarrow k < n \Rightarrow$ reduce redundant information

Apply PCA to $\mathbf{X}$ with $k = 40$

Perform PCA, i.e., solving

$$\underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}, \Theta}{\text{minimize}} \quad \|\mathbf{X} - \mathbf{A}\Theta\|_F^2$$

We get the extracted features (i.e., $\mathbf{A} \in \mathbb{R}^{d \times k}$)



where we resize each column of $\mathbf{A}$ (of dimension $d = 10304$) to a 92×112 image and show it.

Reconstructing the Face Image

$$\tilde{x}_i \approx \mathbf{A}\boldsymbol{\theta}_i = \sum_{j=1}^k \theta_i[j] \mathbf{A}_j$$

Diagram illustrating the reconstruction of a face image x_i as a linear combination of columns in matrix $\mathbf{A}$.

The matrix $\mathbf{A}$ is composed of columns $\mathbf{A}_1, \dots, \mathbf{A}_j, \dots, \mathbf{A}_k$.

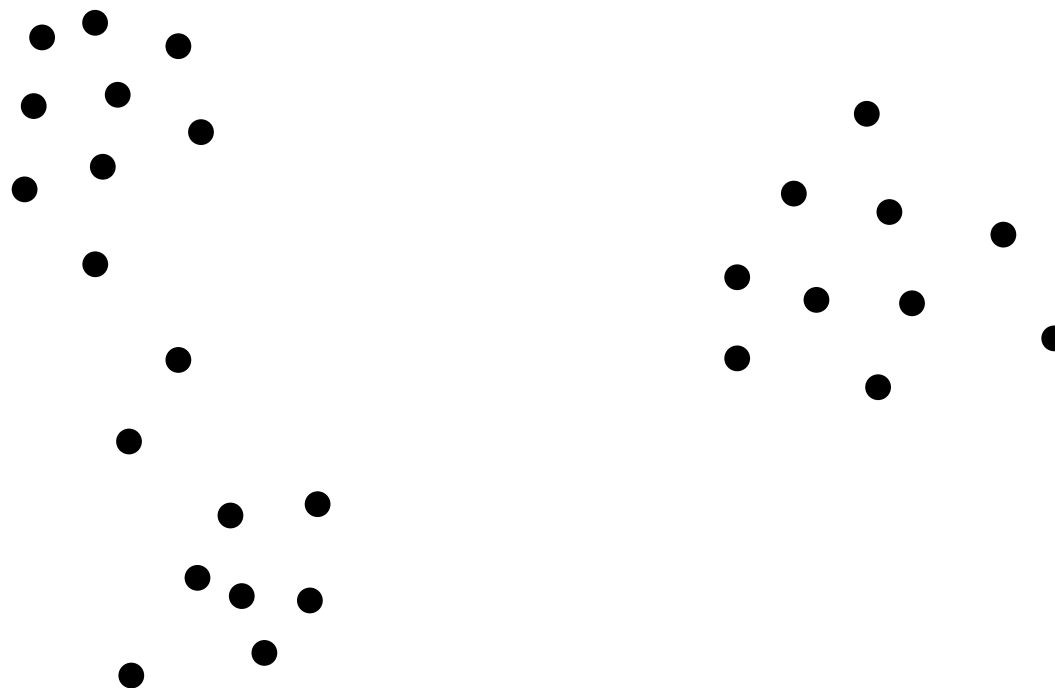
The coefficient vector $\boldsymbol{\theta}_i$ is represented by a vertical stack of colored squares, with the handwritten expression $\boldsymbol{\theta}_i = \begin{bmatrix} \theta_i[1] \\ \vdots \\ \theta_i[j] \\ \vdots \\ \theta_i[k] \end{bmatrix}$.

- ▶ Each face image x_i can be interpreted as a linear combination of the columns in $\mathbf{A}$.
- ▶ The importance of each feature is implied in $\boldsymbol{\theta}_i$.
- ▶ $x_i \in \mathbb{R}^d$ ($d = 10304$ here) has been nicely represented by a k -dimensional vector $\mathbf{A}\boldsymbol{\theta}_i$ ($k = 40$ here) — Dimensionality reduction.

PCA — Continued

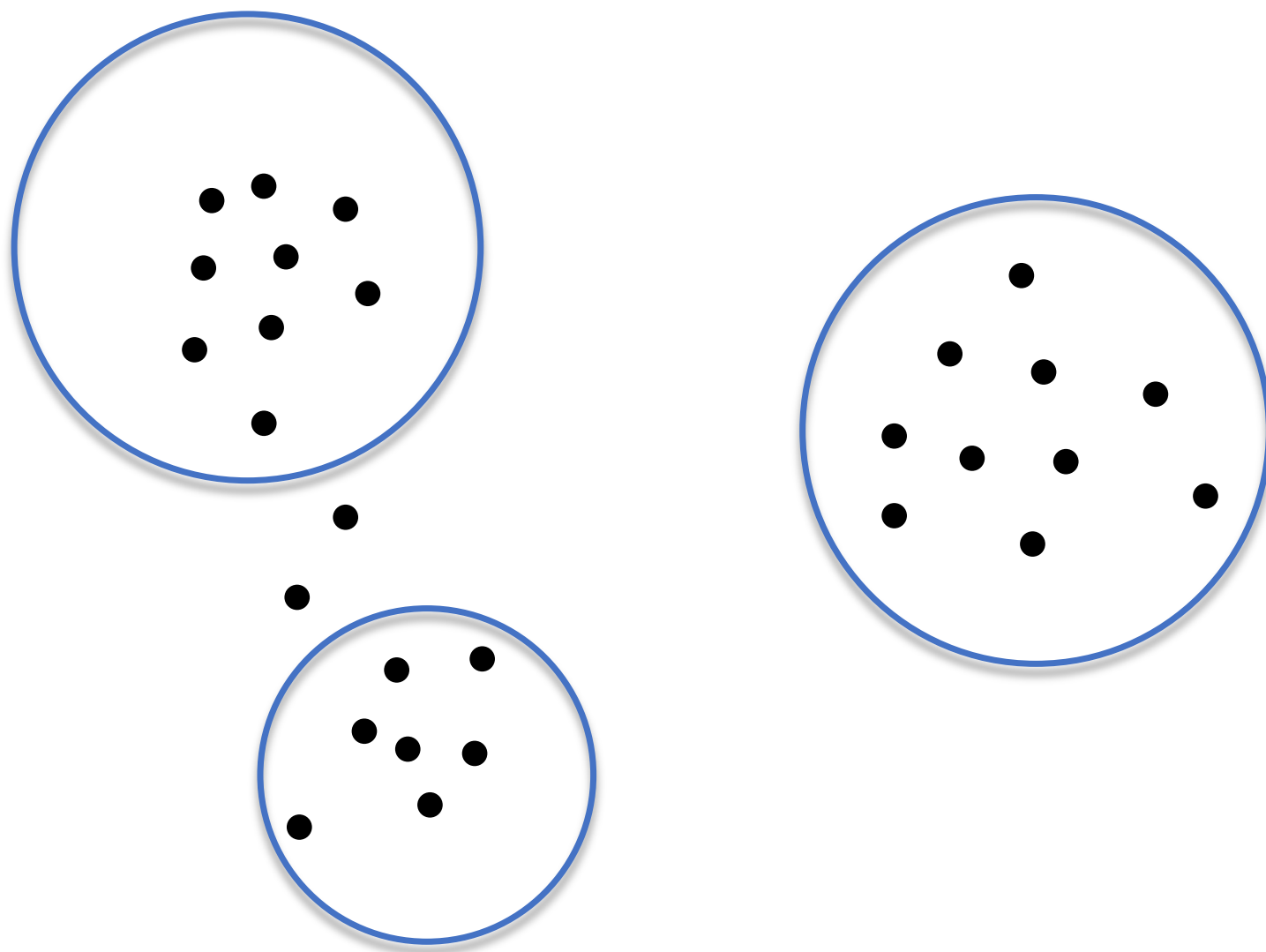
Clustering: k-means

The Unlabeled Samples



- ▶ Unlabeled samples.
- ▶ **Task:** Assign data to several groups / clusters.

Clustering



- This task is very different from classification.

Clustering: Formal Definition

- ▶ Given **unlabeled** data samples $x_1, \dots, x_n \in \mathbb{R}^d$.

Clustering: Assign the unlabeled data to **disjoint subsets** called **clusters**.
The principle is that **points in the same cluster are more similar to each other than points in different clusters**.

↑
depends on how
to def similarity

A clustering algorithm can be represented by a cluster mapping, which is a function

$$C: \{1, \dots, n\} \rightarrow \{1, \dots, k\}$$

Handwritten notes: $x_1, \dots, x_n$ above the domain; k clusters below the codomain.

where k is the number of clusters.

Intuition for performing clustering:

- ▶ Define certain distance functions to **measure similarity**.
- ▶ Compute pair-wise distances and put a threshold to assign clusters.

Clustering vs. Classification

- ▶ **Clustering:** Unsupervised, unlabeled data, assign data into clusters without recognizing them. For example, we just cluster dog and cat without recognizing whether x is dog or cat.
- ▶ **Classification:** Supervised, labeled data, recognize the given data, i.e., tell whether x is dog or cat.

k-means Clustering

k-means

One natural method for clustering is **k-means**, which is to cluster by determining a center of each group.

k-means criterion: Choosing $\mathcal{C}$ to **minimize**

$$\min_{\mathcal{C}} W(\mathcal{C}) = \sum_{j=1}^k \sum_{i:\mathcal{C}(i)=j} \|x_i - \mu_j\|_2^2$$

Sum over all k clusters.

Sum over $x_i \in$ cluster j using

where

$$\mu_j := \frac{1}{n_j} \sum_{i:\mathcal{C}(i)=j} x_i$$

mean / center / Centriod
of cluster j

$$n_j = |\{i : \mathcal{C}(i) = j\}|$$

of data points in cluster j

the current $\mathcal{C}$.

k-means

One natural method for clustering is **k-means**, which is to cluster by determining a center of each group.

k-means criterion: Choosing $\mathcal{C}$ to **minimize**

$$\min_{\mathcal{C}} W(\mathcal{C}) = \sum_{j=1}^k \sum_{i:\mathcal{C}(i)=j} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|_2^2$$

where

$$\boldsymbol{\mu}_j := \frac{1}{n_j} \sum_{i:\mathcal{C}(i)=j} \mathbf{x}_i \quad n_j = |\{i : \mathcal{C}(i) = j\}|$$

- In k-means, the number of clusters k is assumed to be known as a **prior**.

k-means

One natural method for clustering is **k-means**, which is to cluster by determining a center of each group.

k-means criterion: Choosing $\mathcal{C}$ to **minimize**

$$\min_{\mathcal{C}} W(\mathcal{C}) = \sum_{j=1}^k \sum_{i:\mathcal{C}(i)=j} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|_2^2$$

where

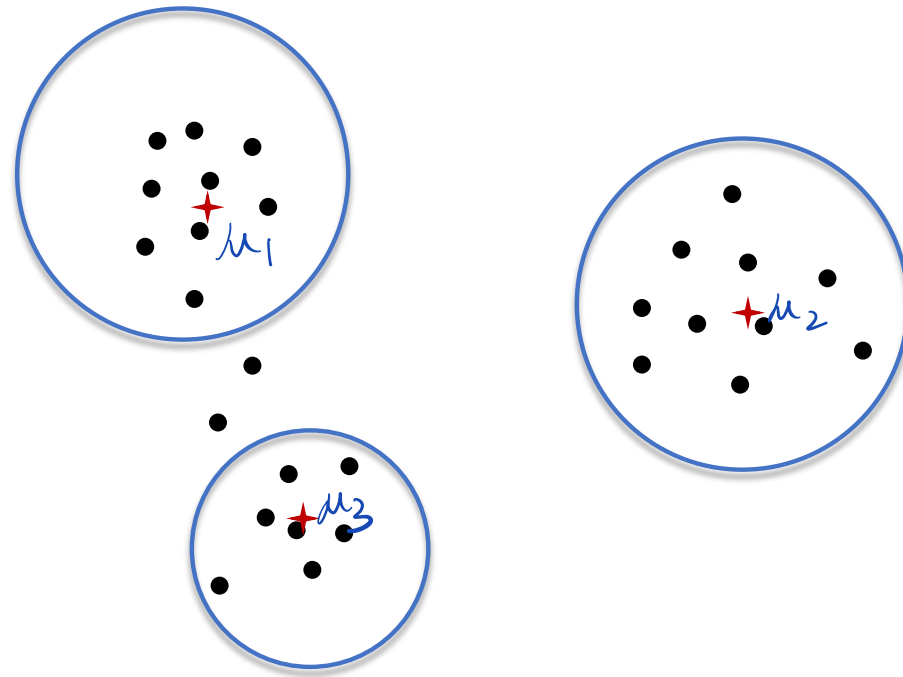
$$\boldsymbol{\mu}_j := \frac{1}{n_j} \sum_{i:\mathcal{C}(i)=j} \mathbf{x}_i \quad n_j = |\{i : \mathcal{C}(i) = j\}|$$

- ▶ In k-means, the number of clusters k is assumed to be known as a **prior**.

Interpretation:

- ▶ k-means: Find k means of the k clusters.
- ▶ Calculate the squared Euclidean distance (ℓ_2 -norm) to measure similarity and dissimilarity.
- ▶ Clustering by finding the assignment that provides small within-cluster distance, i.e., minimizing $W(\mathcal{C})$.

k-means: Interpretation



- Intuition: Find k balls (due to Euclidean distance) to enclose each cluster.

How many possible clusters?

How many cluster maps $\mathcal{C}$ do we need to consider?

$S(n, k) = \#$ of possible clusterings of n objects into k clusters

$$= \frac{1}{k!} \sum_{j=1}^k (-1)^{k-j} C_k^j j^n$$

where $C_k^j = \frac{k(k-1)\cdots(k-j+1)}{j(j-1)\cdots 1}$.

Examples:

► $S(10, 4) = 34105$.

► $S(19, 4) \approx 10^{10}$.

Thus, solving **exactly** a k-means problem is **almost impossible**.

Minimizing The k-means Criterion

- ▶ No existing (provably) efficient method for solving it.
- ▶ It can be solved exactly (by enumerating) in time $\mathcal{O}(n^{dk+1} \log n)$, which is **exponential time**.
- ▶ This is completely impractical unless both d and k are extremely small.
 - E.g., $d = 2$, $k = 3$ already results in $\mathcal{O}(n^7 \log n)$ solving time.

Minimizing the k-means criterion is a **combinatorial optimization** problem, which is **NP-hard**.

In practice, we resort to **iterative, suboptimal** algorithms—**k-means algorithm**.

Reformulation of k-means

Recall k-means is to

$$\mathcal{C}^* = \operatorname{argmin}_{\mathcal{C}} \sum_{j=1}^k \sum_{i: \mathcal{C}(i)=j} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|_2^2.$$

For **fixed** $\mathcal{C}$

$$\boldsymbol{\mu}_j = \operatorname{argmin}_{\mathbf{m}} \sum_{i: \mathcal{C}(i)=j} \|\mathbf{x}_i - \mathbf{m}\|_2^2. \rightarrow \frac{\sum_{i: \mathcal{C}(i)=j} \mathbf{x}_i}{n_j}$$

Thus, we can write k-means **equivalently**

$$\mathcal{C}^* = \operatorname{argmin}_{\mathcal{C}, \{\mathbf{m}_j\}_{j=1}^k} \sum_{j=1}^k \sum_{i: \mathcal{C}(i)=j} \|\mathbf{x}_i - \mathbf{m}_j\|_2^2.$$

- ▶ We now have **two types** of decision variables: 1) **The clustering map**.
2) **The means/centers/centroids** $\{\mathbf{m}_j\}$.
- ▶ This reformulation gives us an **alternating minimization** framework.

k-means Algorithm

$$\mathcal{C}^* = \operatorname{argmin}_{\mathcal{C}, \{\mathbf{m}_j\}_{j=1}^k} \left\{ W(\mathcal{C}, \{\mathbf{m}_j\}_{j=1}^k) := \sum_{j=1}^k \sum_{i: \mathcal{C}(i)=j} \|\mathbf{x}_i - \mathbf{m}_j\|_2^2 \right\}.$$

- ▶ In the formulation above, we have **two** types of decision variables.
- ▶ Recall that optimization algorithm almost always follows the idea: Dividing the original problem into a set of simpler subproblems.
- ▶ Solving the k-means problem for both the clustering map $\mathcal{C}$ and the centers $\{\mathbf{m}_j\}$ is hard. **Can we just solve one of them at one time? It could be much simpler than solving the two variables simultaneously.**

k-means Algorithm

$$\mathcal{C}^* = \operatorname{argmin}_{\mathcal{C}, \{\mathbf{m}_j\}_{j=1}^k} \left\{ W(\mathcal{C}, \{\mathbf{m}_j\}_{j=1}^k) := \sum_{j=1}^k \sum_{i: \mathcal{C}(i)=j} \|\mathbf{x}_i - \mathbf{m}_j\|_2^2 \right\}.$$

- ▶ In the formulation above, we have **two** types of decision variables.
- ▶ Recall that optimization algorithm almost always follows the idea: Dividing the original problem into a set of simpler subproblems.
- ▶ Solving the k-means problem for both the clustering map $\mathcal{C}$ and the centers $\{\mathbf{m}_j\}$ is hard. **Can we just solve one of them at one time? It could be much simpler than solving the two variables simultaneously.**

Alternating minimization algorithm:

- ▶ **Fix $\mathcal{C}$** , update $\{\mathbf{m}_j\}_{j=1}^k$ to minimize $W(\mathcal{C}, \{\mathbf{m}_j\}_{j=1}^k)$.
- ▶ **Fix $\{\mathbf{m}_j\}_{j=1}^k$** , update $\mathcal{C}$ to minimize $W(\mathcal{C}, \{\mathbf{m}_j\}_{j=1}^k)$.

k-means Algorithm

Key advantage: Each subproblem can be solved very easily.

k-means Algorithm

Key advantage: Each subproblem can be solved very easily.

Use t to denote the iteration number.

k-means algorithm

- Fix $\mathcal{C} = \mathcal{C}^{(t)}$, the update of $\{\mathbf{m}_j\}_{j=1}^k$ is

$$\mathbf{m}_j^{(t+1)} = \frac{1}{n_j} \sum_{i: \mathcal{C}^{(t)}(i)=j} \mathbf{x}_i, \quad j = 1, \dots, k.$$


- Fix $\{\mathbf{m}_j\}_{j=1}^k = \{\mathbf{m}_j^{(t+1)}\}_{j=1}^k$, the update of $\mathcal{C}$ is

$$\mathcal{C}^{(t+1)}(i) = \operatorname{argmin}_{1 \leq j \leq k} \|\mathbf{x}_i - \mathbf{m}_j^{(t+1)}\|_2^2, \quad i = 1, \dots, n.$$

- Iterate the above two steps until convergence.

k-means Algorithm

Key advantage: Each subproblem can be solved very easily.

Use t to denote the iteration number.

k-means algorithm

- Fix $\mathcal{C} = \mathcal{C}^{(t)}$, the update of $\{\mathbf{m}_j\}_{j=1}^k$ is

$$\mathbf{m}_j^{(t+1)} = \frac{1}{n_j} \sum_{i: \mathcal{C}^{(t)}(i)=j} \mathbf{x}_i, \quad j = 1, \dots, k.$$

- Fix $\{\mathbf{m}_j\}_{j=1}^k = \{\mathbf{m}_j^{(t+1)}\}_{j=1}^k$, the update of $\mathcal{C}$ is

$$\mathcal{C}^{(t+1)}(i) = \operatorname{argmin}_{1 \leq j \leq k} \|\mathbf{x}_i - \mathbf{m}_j^{(t+1)}\|_2^2, \quad i = 1, \dots, n.$$

- Iterate the above two steps until convergence.

In the update of $\mathcal{C}^{(t+1)}(i)$, each data sample $\mathbf{x}_i$ is assigned to exactly one cluster, even though it can be assigned to more than one cluster.

See <https://www.youtube.com/watch?v=5I3Ei69I40s> for an illustration of the process.

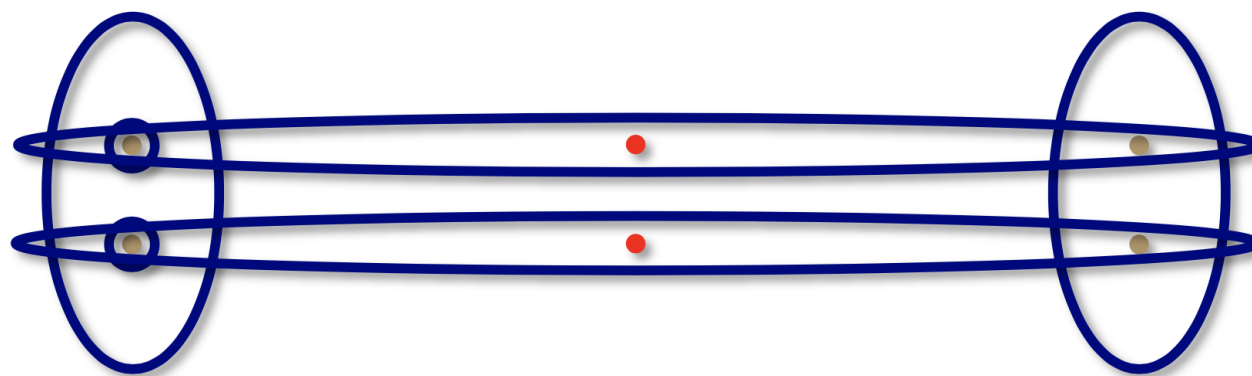
k-means Algorithm: Properties

- ▶ Since each step optimizes $W(\mathcal{C})$ and there only exists finite number of partitioning, so the algorithm usually converges to a (local) optimum in finite number of iterations.
- ▶ No guarantee for convergence to global optimum.
- ▶ Thus, initialization can be important.

Initialization

The algorithm is typically initialized by setting each m_j to be a random point in the dataset, i.e., random initialization.

However, depending on the initialization, the algorithm can get stuck in a local minimum.

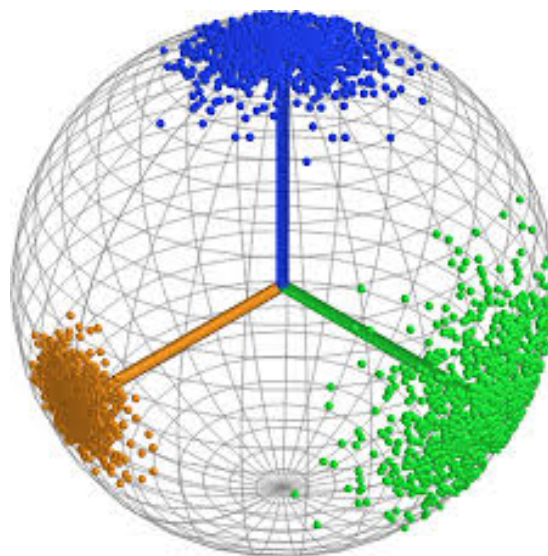


One can avoid this by:

- ▶ Repeating for several random initializations, then check which $W(\mathcal{C})$ is the smallest (very common choice in practice).
- ▶ Initialize by sequentially selecting random points in the dataset, but with a probability depending on how far the point is from the already selected m_j . This technique leads to the k-means ++.

Remarks

- ▶ k-means algorithm was originally developed at Bell Labs as an approach to **vector quantization** in signal processing.
- ▶ If we replace the squared ℓ_2 -norm with the **ℓ_1 -norm** distance in the definition of $W(\mathcal{C})$:
 - The center of each region is actually calculated via the **median**.
 - Results in **k-medians clustering**, which can be more **robust**.
- ▶ If we change the Euclidean squared ℓ_2 -norm distance to the **cosine similarity**, we get the **spherical k-means**.



Model Selection for k-means

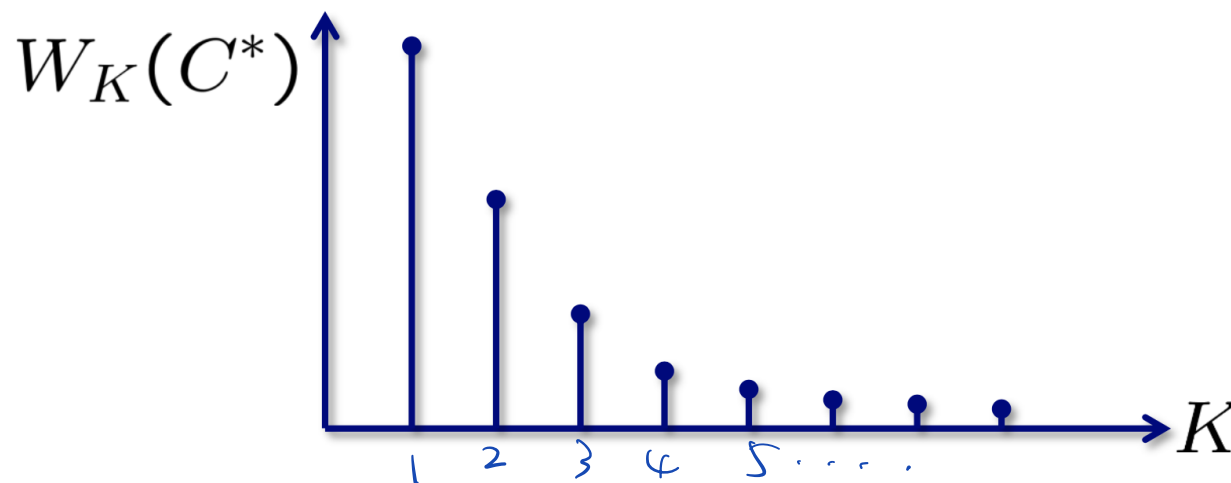
What is model selection used for?

Model Selection for k-means

What is model selection used for? Selecting hyper-parameters.

Sometimes in practice, k is **not** known as a prior, then it becomes a **hyper-parameter**.

Let $W_k(C^*)$ be the optimal loss based on # of clusters k .



Suppose the “correct” number of clusters is k^* , we expect

► for $k < k^*$, $W_k(C^*) - W_{k-1}(C^*)$ will be **large**.

► for $k > k^*$, $W_k(C^*) - W_{k-1}(C^*)$ will be **small**.

→ $k = 4$

This suggests choosing k to be near the “**knee**” of the curve.

Matrix Factorization Perspective for k-means

k-means can also be viewed as a special matrix factorization:

$$X = [x_1 \cdots x_n]$$

$$X \approx BC,$$

$$C = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

where

$$B = [m_1, \dots, m_k]$$

and C has exactly one “1” per column, i.e., unit vector.

- **Interpretation:** Assign each sample, column of X , to one cluster, represented by the “1” element in the corresponding column in C .
- Learning problem:

$$\min_{B, C} \|X - BC\|_F^2, \quad \text{subject to} \quad C \text{ has exactly one “1” per column.}$$

⇒ This is the end of unsupervised learning